Coupling between Perception and Manipulation: Learning to Grasp Objects in Highly Cluttered Environments

Cognitive Robotics Course
https://rugcognitiverobotics.github.io

Assignment Overview

Service robots use perception to understand their surroundings and guide interactions with people and objects. To assist with everyday tasks, a robot must determine *how to grasp and manipulate objects in different situations*. For example, clearing a table requires a robot to connect perception with action: observe a scene, identify a feasible grasp, move to the object, and place it in a basket. Clutter makes this task difficult because objects can occlude one another, restrict access to grasp points, and move during manipulation. In this assignment, you will investigate how learned grasp predictions translate into successful pick-and-place actions.

The provided PyBullet environment contains a dual-arm robot with two UR5e arms and Robotiq 2F-140 grippers, a worktable, two baskets, and simulated objects from the YCB dataset [2]. The main perception input is a fixed, top-down RGB-D camera: an *eye-to-hand* configuration in which the camera observes the workspace from outside the robot. Figure 1 shows the setup.

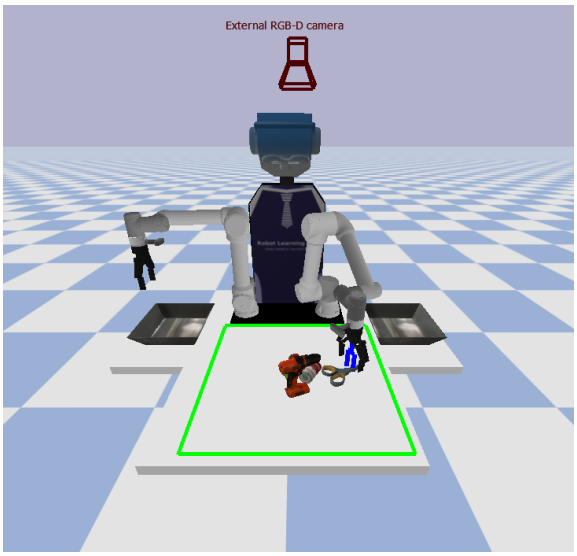


Figure 1: Our experimental setup consists of a table, two baskets, a dual-arm robot, and objects from the YCB dataset. The green rectangle marks the robot's workspace, and the red camera indicates the pose of the fixed top-down RGB-D camera used for grasp prediction.

Traditional grasping approaches often rely on prior knowledge of an object's shape or pose. Obtaining this information can be difficult for previously unseen objects. *Object-agnostic* methods instead predict grasps from visual features without requiring an object identity or an object-specific grasp model at inference time. The supplied baseline uses GR-ConvNet [1] to predict image-space grasp quality, angle, and opening width from RGB-D input. The system selects a candidate, converts it to a robot-frame grasp pose using depth and camera calibration, and executes a pick-and-place action. This is an *object-agnostic* approach: grasp prediction does not require an object-specific grasp model or an object identity label at inference time.

By completing this assignment, you should be able to:

- Explain the complete perception-to-action pipeline, including grasp representation, coordinate transforms, and execution.
- Integrate and compare at least one more grasping method in isolated, packed, and pile scenarios (Fig. 3).
- Evaluate robustness to domain randomization and distinguish perception failures from execution failures (Fig. 2).
- Design reproducible experiments and support conclusions with quantitative results and representative failure cases.

Note that you need to use this setup for your final course project.

Your tasks

This assignment comprises two parts, each worth 50% of your grade. **For the first part**, read the GR-ConvNet paper, inspect the provided code, and evaluate the baseline in all three scenarios. Explain image preprocessing, the network's

grasp representation, candidate selection, image-to-robot coordinate transforms, and gripper opening limits. Describe how the robot selects an arm, executes a grasp, and checks grasp and placement success. Relate the observed performance to scene complexity and failure modes. Note that the basket locations are known, while grasp poses must be predicted from the observed scene.

For the second part, select another grasping approach, justify your choice, and integrate it into the system. Evaluate it in the isolated, packed, and pile scenarios, and compare its results with GR-ConvNet.

You may choose a depth-based method such as GG-CNN or GG-CNN2 [3], a 3D method such as VGN [4], or another suitable deep-learning or data-driven approach. For point-cloud methods, explain how you obtain 3D input from calibrated depth images; the Open3D library is one possible tool. **We strongly recommend using pre-trained models; training from scratch is not required.**

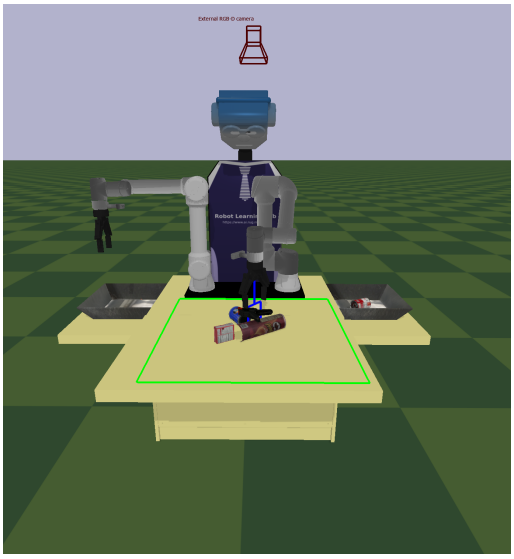


Figure 2: An example of domain randomization, which varies table and floor colors, object textures, lighting, and RGB brightness. The main table, basket platforms, and robot stand share the same sampled table color.

Then evaluate **both methods with domain randomization enabled** and compare each with its standard-appearance results. For this assignment, domain randomization changes appearance only: table and floor colors, object textures, lighting, and RGB brightness (Fig. 2). Settings are sampled once per packed or pile round, or per isolated-object episode, and remain fixed throughout that episode. Depth, segmentation, camera geometry, object geometry, and physics are unchanged. Discuss which changes affect each method's performance. Note that testing a fixed model under randomized appearance measures robustness; it does not train the model or establish improved transfer to a real robot.

To evaluate each approach, perform **10 rounds per scenario and per appearance condition**. Each packed or pile round starts with **five objects**. In the isolated scenario, one object is placed at a randomized pose. Note that `-runs=10` in the supplied isolated runner performs ten sweeps through the configured object list, with one episode per object in each sweep.

In this assignment, the primary task metric is **successful placement**, called *manipulation success rate* in the supplied results. A lift alone does not complete the task. Report $\texttt{success_rate} = \frac{\text{number of successful basket placements}}{\text{number of attempts}}$ as a percentage, together with the numerator and denominator. The separate grasp-success metric can help distinguish grasping failures from transport and placement failures.

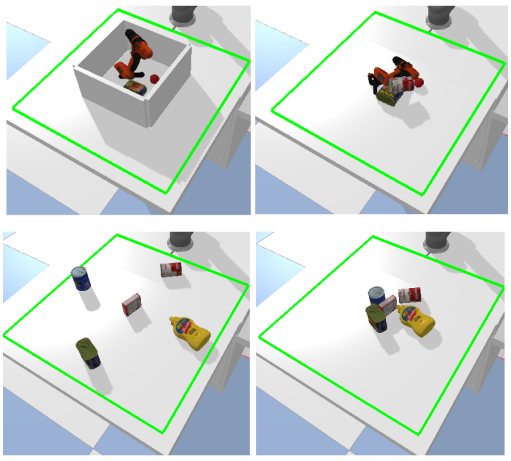


Figure 3: Generating random scenes: objects are dropped into temporary walls to form a pile (*top row*); objects are brought close together to form a packed scene (*bottom row*).

To create a pile, the simulator drops objects into temporary walls above the table, lets them settle, and removes the walls before grasping. For a packed scene, it places objects around a central object and moves the surrounding objects inward. These procedures are illustrated in Fig. 3.

For packed and pile scenes, also report the **average percentage of objects successfully cleared**: successful basket placements divided by the five initial objects, averaged over all rounds. Objects pushed or dropped off the table do not count as successfully cleared. Continue a round until the workspace is empty or **four consecutive attempts fail to place an object in a basket**. A successful placement resets the failure count. If needed, you can set it in the command line by `-attempts=6` explicitly; the command-line default is 4. Note that the system automatically reports a summary of the obtained results in the "`results`" folder, and the prediction of the network is visualized and saved in the "`network_output`" folder.

What we offer for this assignment

- Detailed installation and experiment instructions are available in `README.md` and the GitHub repository. For faster experiments, run with `-gui=false`. You can enable camera recordings with `-record_episodes=true` and review them using `view_episodes.py`.

- The system saves experiment summaries in `results/`. Network prediction plots are saved in `network_output/` when `-save-network-output=true`. Keep outputs from different methods and appearance conditions identifiable in your experiment records.

Policies

- **AI-assisted coding is permitted.** In the report's acknowledgments, name the tools you used and explain how you used them. You remain responsible for understanding the code and verifying implementations, results, and references.
- You may discuss ideas and approaches with other students, but please write your code individually. **Do not copy code or text from other students**. Explain each group member's contribution in the report.
- **You are not allowed** to use the provided code in other projects, including partial reuse, without prior permission. If you wish to publish results or use this framework outside the course, contact Hamidreza Kasaei (`hamidreza.kasaei@rug.nl`) to discuss the proposed use. The final course project is an intended use of this setup.

Submission

Submit a PDF report of **up to four pages, plus a one-page appendix**, using the IEEE conference format. Explain the perception-to-action pipeline and both methods, document the experimental settings and counting conventions, and compare results across all scenarios and both appearance conditions. Discuss representative failure cases, domain randomization, and limitations supported by your evidence.

At the end of the report, include an **Authors' Contributions** section explaining how the work was divided and what each author contributed. All authors are expected to contribute equally. Include the AI-tool disclosure in the acknowledgments when applicable. Submit the PDF as `group_number_prj1.pdf`.

⚖ Do not delete your code, configurations, or raw results after submitting the report. We may ask you to provide your implementation and experiment outputs to verify the reported findings.

References

[1] S. Kumra, S. Joshi, and F. Sahin, "Antipodal robotic grasping using generative residual convolutional neural network," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9626–9633, 2020. doi:10.1109/IROS45743.2020.9340777.

[2] B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar, "Benchmarking in manipulation research: Using the Yale-CMU-Berkeley object and model set," *IEEE Robotics & Automation Magazine*, vol. 22, no. 3, pp. 36–52, 2015. doi:10.1109/MRA.2015.2448951.

[3] D. Morrison, P. Corke, and J. Leitner, "Closing the loop for robotic grasping: A real-time, generative grasp synthesis approach," in *Robotics: Science and Systems (RSS)*, 2018. Paper and reference implementation.

[4] M. Breyer, J. J. Chung, L. Ott, R. Siegwart, and J. Nieto, "Volumetric grasping network: Real-time 6 DOF grasp detection in clutter," in *Proceedings of the 2020 Conference on Robot Learning*, *Proceedings of Machine Learning Research*, vol. 155, pp. 1602–1611, 2021. Paper and citation details.